Documentation technique – P7-FSJA

Projet	MicroCRM / P7-FSJA
Auteur	Benjamin LUCAS
Date	09/04/2026
Option choisie	Option B – Scénario Orion

1. Introduction

Le projet P7 industrialise une application MicroCRM composée d'un backend Spring Boot 3.2.5 en Java 17 et d'un frontend Angular 17. La chaîne cible couvre le build, les tests, l'analyse statique SonarCloud, la publication d'images Docker et un premier niveau d'observabilité avec ELK.

- **Contexte :** mise en place d'un pipeline CI/CD reproductible pour une application full stack
- **Objectifs :** fiabiliser les builds, identifier les mauvaises pratiques, éviter la régression, automatiser les tests, mesurer la qualité, préparer le déploiement et formaliser un plan de sécurité
- **Technologies principales :** Spring Boot, Gradle, Angular, Node.js 20, SonarCloud, Docker, GitHub Actions, Caddy, ELK

2. Étapes de mise en œuvre du pipeline CI/CD

2.1 Structure du pipeline

La CI est déclenchée sur push, pull request vers main et lancement manuel. Le CD est déclenché automatiquement après une CI réussie sur la branche principale.

Étape	Contenu	Pourquoi
CI – job backend	checkout, Java 17, correction des fins de ligne, chmod gradlew, ./gradlew clean build	Valider la compilation backend et les tests Gradle.
CI – job frontend	checkout, Node.js 20, npm ci, npm run build, npm run test:ci	Valider le front Angular et produire la couverture.
CI – job SonarCloud	génération JaCoCo + LCOV puis analyse SonarCloud	Consolider la qualité de code et la couverture.
CD – publication	workflow_run après CI verte, build Docker multi-stage, push vers GHCR	Fournir un déploiement continu simple et traçable.

2.2 Scripts d'automatisation

- **Backend :** ./gradlew clean build --no-daemon
- **Couverture backend :** ./gradlew clean test jacocoTestReport --no-daemon
- **Frontend :** npm ci, npm run build, npm run test:ci
- **Lancement local applicatif :** docker compose up --build
- **Lancement observabilité :** docker compose -f docker-compose-elk.yml up -d

2.3 Reproductibilité

- Le pipeline peut être relancé manuellement via workflow_dispatch.

- Les variables de projet Sonar sont stockées dans GitHub Variables ; le token Sonar dans GitHub Secrets.
- Le CD utilise le GITHUB_TOKEN natif pour publier dans GitHub Container Registry.

3. Plan de conteneurisation et de déploiement

3.1 Dockerfiles

Le Dockerfile utilise une stratégie multi-stage : Node 20 Alpine pour construire le front, Gradle JDK 17 pour le backend, puis deux images runtime légères basées sur Alpine 3.19. Le frontend est servi par Caddy et le backend par OpenJDK 17 JRE headless.

- Le target front expose les ports 80 et 443 et sert les fichiers Angular compilés.
- Le target back expose le port 8080 et lance le JAR Spring Boot.
- Un target standalone permet de réunir front et back avec supervisor pour certains tests locaux.

3.2 docker-compose.yml

Le docker-compose principal définit deux services : back et front. Le front dépend du back et l'ensemble permet un démarrage local complet avec une seule commande.

Service	Rôle	Ports
back	API Spring Boot	8080:8080
front	Application Angular servie par Caddy	80:80 ; 443:443

4. Plan de testing périodique

4.1 Types de tests automatisés

- Tests backend Gradle / JUnit à chaque push, pull request et relance manuelle.
- Tests frontend Angular / Karma en mode CI avec génération de couverture LCOV.
- Analyse SonarCloud sur le code Java et TypeScript après validation des jobs build/test.

Critères de réussite : build vert, tests passants, couverture générée et analyse SonarCloud sans blocage du Quality Gate. Critères d'alerte : échec d'un job, baisse marquée de couverture, apparition de règles critiques supplémentaires ou erreur applicative remontée par ELK.

4.2 Fréquence d'exécution

- Sur push vers main : build, tests et analyse SonarCloud.
- Sur pull request vers main : build, tests et analyse SonarCloud avant validation du merge.
- Après réussite de la CI sur main : déclenchement du déploiement continu et publication des images Docker.

- Nightly build recommandé en amélioration future pour vérifier la stabilité globale et l'évolution des dépendances.
- Avant release ou démonstration : relancer l'exécution complète build + tests + couverture + SonarCloud, puis vérifier la reconstruction des images et le démarrage applicatif.

4.3 Objectifs des tests

- Prévenir les régressions fonctionnelles visibles à la compilation et aux tests.
- Mesurer la couverture et la qualité du code côté front et côté back.
- Bloquer la chaîne de déploiement si la CI échoue.

5. Plan de sécurité

5.1 Résultats SonarCloud

Mesure	Valeur	Commentaire
Vulnérabilités	0	Aucune vulnérabilité ouverte dans l'export Sonar.
Bugs	2	À surveiller, mais moins structurants que les problèmes critiques du front.
Code smells	49	Dette technique modérée, concentrée surtout côté frontend.
Règles critiques ouvertes	16	14 sur S4123, 1 sur S7059, 1 sur S1186.
Quality Gate	OK	Le projet passe le seuil défini, malgré une couverture encore basse.

Mesure	Valeur	Commentaire
Couverture globale	37,4 %	Insuffisant pour un module métier durablement sécurisé.
Duplication	2,5 % (28 lignes)	Faible à l'échelle du projet, mais localisée dans les entités Java.
Complexité totale	91	À rapprocher des zones de code les plus sensibles.
Complexité cognitive	18	Principalement portée par les composants de détail front.

5.2 Duplications et zones à forte complexité

Fichier	Complexité	Complexité cognitive	Couverture	Commentaire
front/src/app/person-details/person-details.component.ts	21	7	26,7 %	Zone la plus sensible du projet.
back/src/main/java/com/openclassroom/devops/orion/microcrm/Person.java	17	1	45,2 %	Duplication et logique métier à surveiller.
front/src/app/organization-details/organization-details.component.ts	11	4	36,0 %	Complexité intermédiaire, couverture encore insuffisante.

Fichier	Complexité	Complexité cognitive	Couverture	Commentaire
back/src/main/java/com/openclassroom/devops/orion/microcrm/Organization.java	11	2	36,4 %	Fichier avec la duplication la plus forte.
front/src/app/organization.service.ts	9	2	12,5 %	Service peu couvert et concerné par les alertes critiques.
front/src/app/person.service.ts	7	2	14,8 %	Même profil de risque que organization.service.ts.

La duplication globale reste limitée à 2,5 %, mais elle se concentre sur les entités Java Organization.java et Person.java. Les zones les plus sensibles du point de vue maintenance et fiabilité se situent côté frontend, en particulier les composants et services de détail.

5.3 Règles critiques violées

- **14 occurrences — typescript:S4123 :** Unexpected await of a non-Promise dans organization.service.ts et person.service.ts
- **1 occurrence — typescript:S7059 :** opération asynchrone placée dans le constructeur de person-details.component.ts
- **1 occurrence — java:S1186 :** méthode de test vide sans justification dans MicroCRMApplicationTests.java

Ces alertes ne signalent pas une faille de sécurité directe, mais elles dégradent la maintenabilité et la robustesse, en particulier sur la gestion asynchrone du frontend Angular.

5.4 Analyse des risques

Risque	Impact	Commentaire
Mauvaise gestion de l'asynchronisme Angular	Moyen	14 occurrences critiques typescript:S4123 sur les services front.
Faible couverture des tests	Élevé	37,4 % de couverture globale : risque de régressions non détectées.
Complexité forte sur certains composants front	Moyen	person-details.component.ts cumule complexité élevée et couverture faible.

Risque	Impact	Commentaire
Duplication localisée dans les entités Java	Faible	2,5 % globalement, concentré sur Organization.java et Person.java.
CORS trop permissif et sécurité applicative légère	Élevé	Acceptable en local, insuffisant pour un environnement exposé.

Au-delà du code applicatif, le pipeline doit aussi rester maîtrisé sur trois points : la bonne séparation des secrets GitHub, la mise à jour régulière des GitHub Actions utilisées et la surveillance des dépendances Docker, npm et Gradle pour limiter l'exposition à des composants obsolètes.

5.5 Plan d'action / remédiation

- **Actions immédiates :** corriger S4123 dans les services Angular, déplacer l'opération asynchrone hors du constructeur, restreindre les origines CORS, externaliser l'URL d'API
- **Actions à court terme :** porter la couverture front à au moins 60 %, ajouter des tests métier backend, réduire la duplication des entités Java si elle progresse,

- **Actions à long terme :** viser une couverture ≥ 80 % sur les zones critiques, renforcer la sécurité applicative backend,

6. Monitoring, métriques et KPI

6.1 Métriques DORA

Métrique	Méthode de calcul	Valeur
Lead Time for Changes	Temps entre le déclenchement CI et la fin du CD réussi correspondant	≈ 6 min 19 s
Deployment Frequency	Nombre de CD réussis observés sur la période disponible	5 déploiements réussis en 8 jours (≈ 4,4/semaine)
MTTR (proxy pipeline)	Temps entre un CD non abouti et le premier CD redevenu vert	≈ 1 h 37
Change Failure Rate	CD non aboutis / déclenchements CD	28,6 % (2/7)

6.2 KPI personnalisés

KPI	Source	Valeur	Commentaire
Temps moyen job backend	GitHub Actions CI	≈ 49 s	Compile le backend et exécute les tests Gradle.
Temps moyen job frontend	GitHub Actions CI	≈ 49 s	Installation, build Angular et tests front.
Temps moyen analyse SonarCloud	GitHub Actions CI	≈ 1 min 29 s	Étape qualité la plus longue de la CI.
Temps moyen pipeline CI complet	GitHub Actions CI	≈ 2 min 18 s	Moyenne sur plusieurs exécutions observées.
Temps moyen pipeline CD	GitHub Actions CD	≈ 3 min 09 s	Construction et publication des images Docker vers GHCR.
Taux de succès pipeline	GitHub Actions	≈ 71,4 %	Lecture stricte : 5 runs aboutis sur 7 déclenchements observés.
Qualité SonarCloud	SonarCloud	Quality Gate OK ;	KPI de qualité composite,

		couverture 37,4 % ; 16 critiques	utile pour le suivi.
Fréquence des erreurs applicatives	Kibana / ELK	0 erreur observée	Aucun log ERROR remonté sur la période d'observation.
Taux de duplication	SonarCloud	2,5 %	Faible, acceptable à ce stade du projet.

6.3 Analyse synthétique du monitoring

Les pipelines sont globalement rapides et bien structurés. Le point fort principal est l'enchaînement cohérent build → tests → couverture → Sonar → publication d'images. La CI complète reste courte malgré l'analyse qualité.

Le monitoring ELK n'a remonté aucune erreur applicative.

La priorité de fiabilité reste le frontend Angular : il combine couverture faible, règles critiques ouvertes et composants relativement complexes. Le backlog qualité doit donc se concentrer sur les services et composants de détail, avant d'élargir l'effort à la sécurité applicative et à la supervision.

Les tableaux de bord minimum à conserver sont GitHub Actions pour le suivi des durées et des statuts de workflow, SonarCloud pour la couverture, la duplication et les issues critiques, et Kibana pour l'exploration des logs applicatifs et la recherche d'erreurs.

7. Plan de sauvegarde des données

7.1 Ce qui doit être sauvegardé

Le projet n'expose pas aujourd'hui de base de données externalisée persistante dans le docker-compose principal. Le périmètre de sauvegarde concerne donc surtout les éléments nécessaires à la reconstruction fiable de l'environnement de build, de déploiement et d'observabilité.

- Dépôt Git et historique des commits stables, branches de référence et tags de release.
- Workflows GitHub Actions, Dockerfile, docker-compose, configuration Caddy / reverse proxy et configuration ELK.
- Inventaire des variables et secrets nécessaires au fonctionnement, conservé dans une documentation d'exploitation séparée sans jamais exposer leur valeur en clair dans le dépôt.
- Artefacts utiles à la remise en service : images publiées sur GHCR, exports de métriques ou de rapports qualité, et, si une base persistante est ajoutée plus tard, les dumps de données associés.

7.2 Procédure de sauvegarde

La sauvegarde doit viser une restauration rapide d'un état stable, et non la seule conservation du code source. Elle repose donc sur une combinaison d'éléments versionnés, d'artefacts publiés et d'une documentation d'exploitation tenue à jour.

- Avant chaque release ou démonstration : identifier le commit stable, poser un tag et vérifier que les images Docker correspondantes sont disponibles dans GHCR.
- Après toute évolution majeure du pipeline : archiver les fichiers de configuration concernés et mettre à jour la documentation d'exploitation, notamment les prérequis, variables d'environnement et dépendances système.
- Si une base persistante est introduite : prévoir un dump quotidien automatisé, complété par un export manuel avant migration ou changement de version.
- Conserver des sauvegardes horodatées et relier chaque sauvegarde à une version applicative identifiable afin de faciliter le retour arrière.

7.3 Procédure de restauration

Scénario d'incident type : échec de déploiement, corruption d'une image publiée, régression sévère détectée après merge ou indisponibilité d'un environnement local/de démonstration. L'objectif est de revenir rapidement à une version stable, reproductible et vérifiable.

- Identifier le dernier commit ou tag stable validé fonctionnellement.
- Récupérer les images Docker correspondantes depuis GHCR ou les reconstruire à partir de ce commit stable.
- Rétablir les variables et secrets requis dans GitHub et dans l'environnement d'exécution, à partir de l'inventaire d'exploitation prévu à cet effet.
- Relancer l'environnement avec docker compose, puis vérifier le bon fonctionnement du backend, du frontend et de la remontée des logs dans Kibana.
- Si une base de données persistante existe, restaurer le dernier dump cohérent

Limitations : en l'absence de base persistante dans l'état actuel du projet, la restauration porte surtout sur la chaîne d'industrialisation et la configuration.

8. Plan de mise à jour

8.1 Mise à jour de l'application

Le maintien en condition opérationnelle de l'application doit couvrir à la fois les frameworks principaux et les dépendances projet. Les mises à jour visent la sécurité, la compatibilité et la stabilité du pipeline.

- Surveiller Angular 17, Spring Boot 3.2.x, Node.js 20, Java 17 ainsi que les dépendances Gradle et npm utilisées par le projet.
- Tester les montées de version sur une branche dédiée avant fusion afin de valider la compilation, les tests, l'analyse SonarCloud et le packaging Docker.
- Vérifier en priorité les bibliothèques exposées côté runtime et les dépendances ayant un impact direct sur la sécurité, la compatibilité navigateur ou le démarrage applicatif.

8.2 Mise à jour du pipeline CI/CD

Le pipeline doit être maintenu comme un composant à part entière de l'application. Les workflows, les images de build et les actions GitHub utilisées doivent être revus régulièrement pour éviter l'obsolescence ou les ruptures de compatibilité.

- Mettre à jour régulièrement les actions GitHub utilisées, notamment checkout, setup-node, setup-java et build-push-action.
- Réévaluer les images Docker de base, en particulier Alpine, Caddy et la pile Elastic, afin de limiter l'exposition à des versions anciennes.
- Conserver des scripts lisibles, versionnés et cohérents avec la configuration réelle du projet pour faciliter les reprises et les audits.

8.3 Fréquence et bonnes pratiques

- Prévoir une revue mensuelle des dépendances et du pipeline, complétée par une revue immédiate en cas de faille critique ou d'évolution majeure d'un composant clé.
- Documenter chaque montée de version significative, ses impacts éventuels et la procédure de retour arrière en cas d'échec.
- Éviter les mises à jour massives non testées : privilégier des incréments maîtrisés, validés par la CI/CD et observés après déploiement.
- Conserver un seuil de qualité stable lors des mises à jour en contrôlant la couverture, les issues SonarCloud et le comportement des images construites.

9. Conclusion

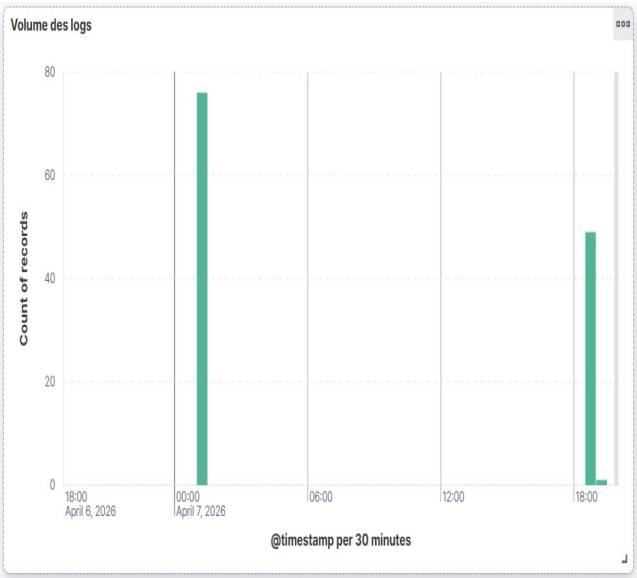
Le pipeline CI/CD mis en place constitue une base solide pour industrialiser MicroCRM. Il apporte déjà des bénéfices concrets : automatisation du build backend et frontend, exécution systématique des tests, mesure de la qualité via SonarCloud, publication des images Docker et premier niveau de supervision avec ELK.

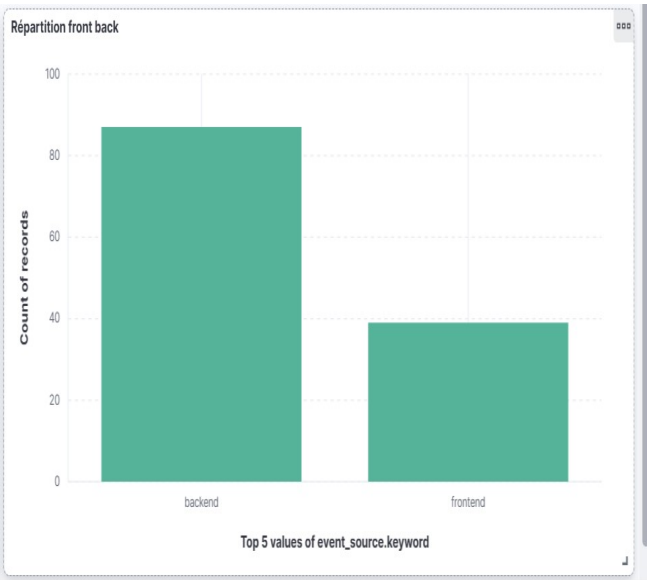
Les indicateurs observés montrent une chaîne globalement rapide et exploitable, mais encore perfectible. Le principal point faible reste le frontend Angular, où se concentrent la faible couverture, la majorité des règles critiques et une part importante de la complexité. À ce stade, la robustesse du projet dépend donc surtout de l'augmentation de la couverture de tests, de la correction des problèmes d'asynchronisme et du renforcement de la sécurité applicative.

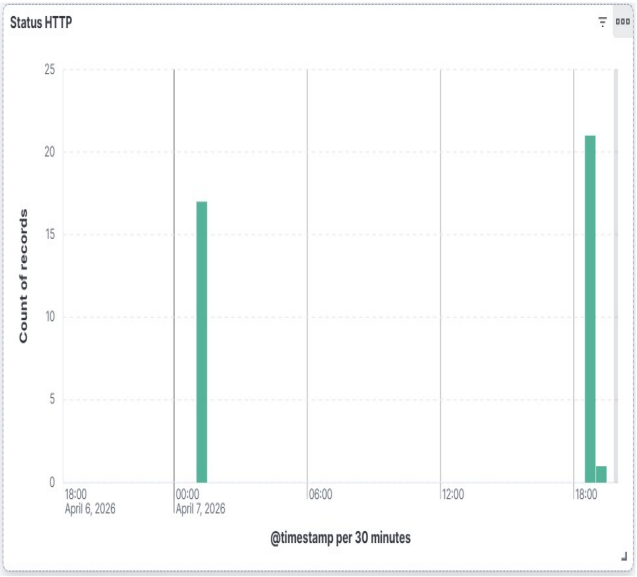
Pour les prochaines itérations, la priorité doit être donnée à trois axes : fiabiliser les zones métier les plus sensibles, formaliser davantage l'exploitation (sauvegarde, restauration, alertes, mise à jour), puis renforcer progressivement le niveau d'exigence qualité. Avec ces améliorations, la solution pourra évoluer vers une chaîne de livraison plus sûre et plus maintenable.

Annexe

Stack ELK

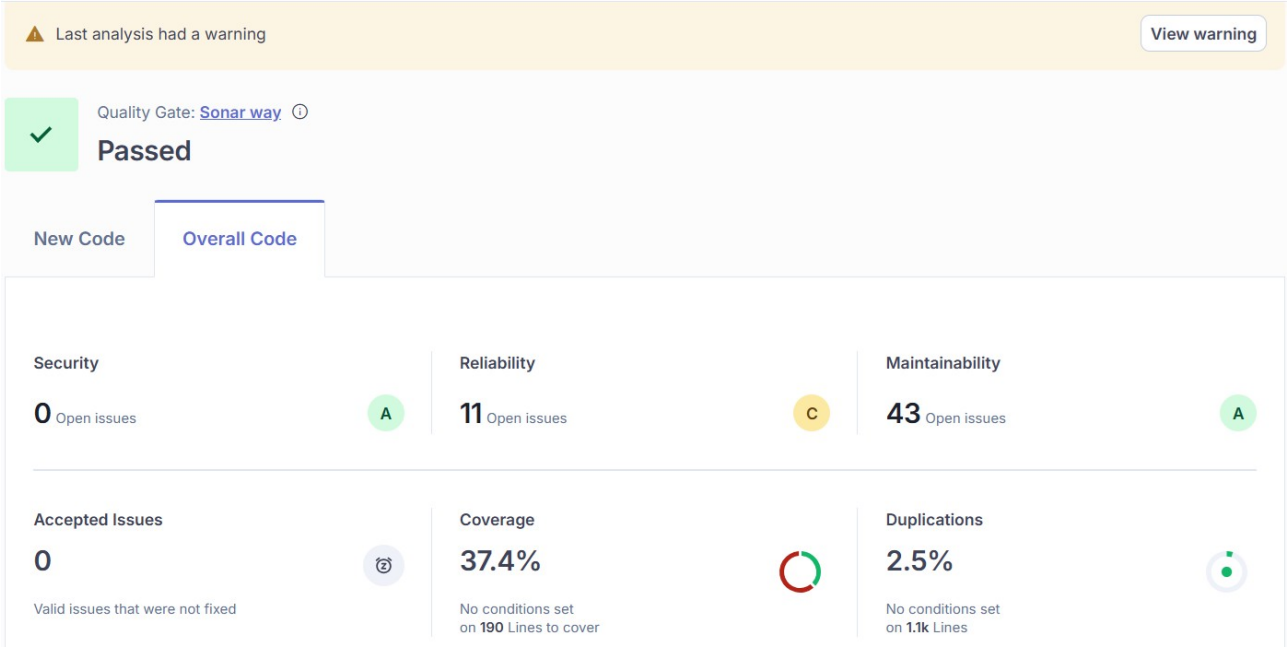




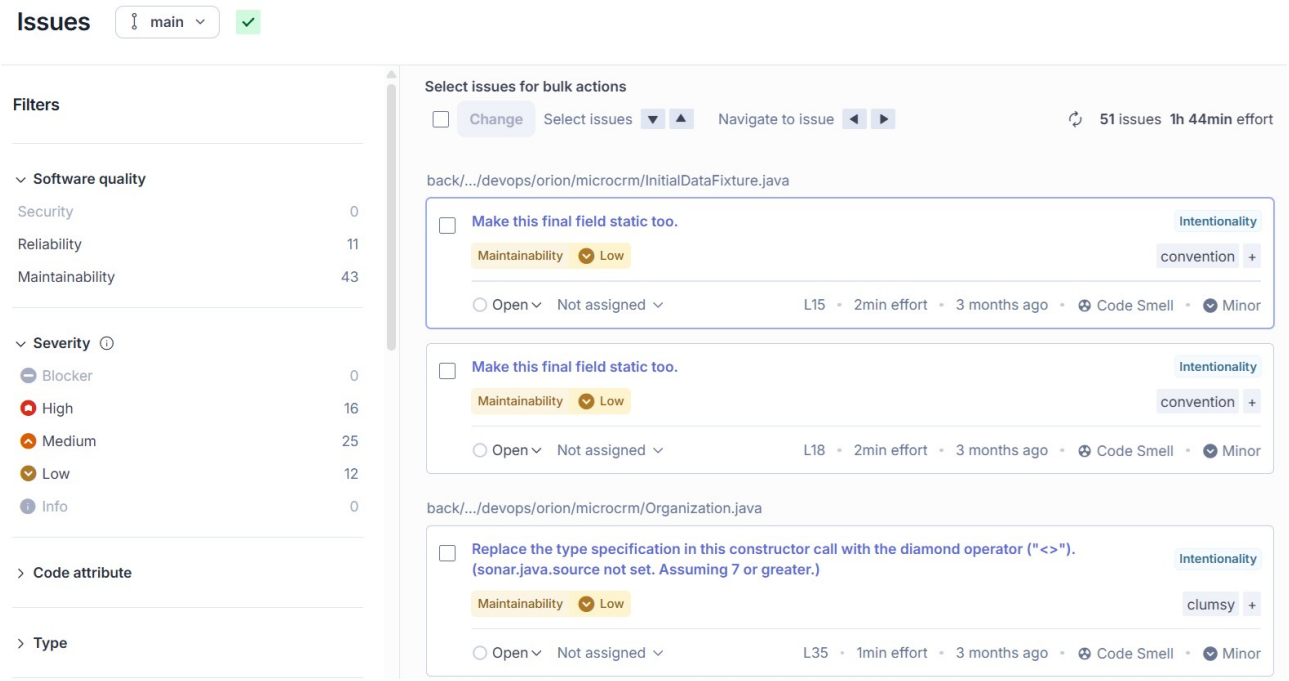


Annexe – Captures SonarCloud et GitHub Actions

SonarCloud - Quality Gate et indicateurs globaux



SonarCloud - Liste des issues détectées



GitHub Actions - Historique des workflows

All workflows

Showing runs from all workflows

Filter workflow runs

14 workflow runs

Workflow | Event | Status | Branch | Actor

CD CD #8: completed by gordinir67		Apr 10, 8:19 PM GMT+2 2m 54s	...
Ajout documentation V1 CI #6: Commit 1bc1087 pushed by gordinir67	main	Apr 10, 8:16 PM GMT+2 3m 22s	...
CD CD #7: completed by gordinir67		Apr 9, 5:41 PM GMT+2 3m 42s	...
Ajout LCOV et jacoco CI #5: Commit db7e35b pushed by gordinir67	main	Apr 9, 5:37 PM GMT+2 3m 20s	...
CD CD #6: completed by gordinir67		Apr 9, 5:25 PM GMT+2 2m 57s	...
Ajout stack ELK CI #4: Commit 8c013b9 pushed by gordinir67	main	Apr 9, 5:22 PM GMT+2 2m 20s	...